



Roque

TECHNICAL ARCHITECTURE

From Natural-Language Trade to On-Chain Settlement

A code-level explanation of Roque's backend, GenLayer judgment layer, autonomous capability model, Latch boundary, keeper/indexer services, and Solidity execution contracts.

01

Interpret

Validators convert a user's words into a constrained exact-input intent.

02

Authorize

A user signature defines the agent's bounded capability.

03

Execute

Sepolia contracts decide whether value can move and settle the trade.

Core invariant

The intelligent layer may propose and coordinate. It cannot bypass deterministic authorization.

Repository: [dotmantissa/roque](https://github.com/dotmantissa/roque)

Snapshot commit: [b029735](https://github.com/dotmantissa/roque/commit/b029735)

Generated: August 14, 2026

Primary network: Ethereum Sepolia (11155111)

GenLayer network: Genlayer Studio Network

Contents

1. Executive Summary	4
1.1 User-visible outcome	4
2. Scope, Sources, and Current Snapshot	4
2.1 Current deployment endpoints	5
2.2 Repository topology	5
3. System Architecture and Trust Boundaries	5
3.1 Data ownership	6
3.2 Network call graph	6
4. Natural-Language Trade Lifecycle	6
4.3 What the intelligent contract asks the model to produce	7
4.4 Deterministic normalization inside GenLayer	7
4.5 Backend validation, persistence, and quote	7
5. Copilot Execution Path	8
5.1 Copilot market swap	8
5.2 Copilot limit order	8
6. Autonomous Mode in Depth	9
6.1 Prerequisites: vault funding	9
6.2 Capability grant and the first signature	9
6.3 Confirm versus direct autonomous execution	10
6.4 Backend autonomous execution	10
6.5 The second signature: per-action agent intent	10
6.6 On-chain authorization order	10
6.7 Autonomous market-swap settlement	11
6.8 Autonomous limit-order creation and later fill	11
7. Solidity Contract Architecture	12
7.1 TestToken and FaucetRouter	12
7.2 Complete pool mesh	12
7.3 DEXRouter	12
7.4 OrderBook state machine	12
7.5 AgentExecutor as the authorization choke point	12
7.6 Administrative trust	13
8. Keeper, Indexer, and Database	13
8.1 Keeper loop	13
8.2 Indexer loop	13



8.3 Intent and trade records	14
8.4 Two backend hosting shapes	14
9. Latch's Role in Autonomous Mode	14
9.1 Exact placement	14
9.2 What Latch contributes	14
9.3 What Latch does not do in this code	15
9.4 Recommended Latch policy for Roque	15
10. Why Rialo/Latch Helps Roque Users	15
10.1 Architectural fit with broader Rialo capabilities	16
11. Security Model and Failure Analysis	16
11.1 Enforced invariants	16
11.2 Threat and failure matrix	17
11.3 Current implementation risks and production work	17
12. Operational Behavior and Configuration	18
12.1 Required server configuration	18
12.2 Retry and polling behavior	18
12.3 Completion semantics by action	18
13. Testing and Verification Evidence	18
13.1 Particularly important tested rejection paths	19
14. Token and Oracle Registry	19
15. Code Map for Maintainers	20
16. External Technical References	20

1. Executive Summary

Roque is an agent-native decentralized exchange demo that deliberately separates language understanding from financial authority. The user's command is interpreted by a GenLayer intelligent contract, coordinated by a shared TypeScript backend, and ultimately accepted or rejected by deterministic Solidity contracts on Ethereum Sepolia. The result is a system where an AI-assisted path can be expressive without making the model, the database, the relayer, or the keeper the final authority over funds.

JUDGMENT	GenLayer intelligent contract Natural language -> normalized intent. Validator consensus handles the non-deterministic LLM call. Holds no funds.
COORDINATION	Shared TypeScript core + relayer Persists requests, reads the finalized interpretation, quotes Sepolia, constructs typed intents, signs as the agent, and broadcasts transactions.
AUTHORIZATION	AgentExecutor on Sepolia Recovers the agent signature and enforces the user's capability, nonce, deadline, Chainlink USD caps, slippage, and vault balance.
EXECUTION	DEXRouter, LiquidityPool, OrderBook Deterministic AMM settlement and oracle-gated resting orders. Events become the canonical settlement record.
GOVERNANCE	Latch on relayer writes Optional RPC credential proxy and policy/audit boundary between locally signed relayer traffic and the upstream Sepolia RPC.

What the shipped system actually guarantees

Autonomous actions can draw only from balances explicitly deposited into the AgentExecutor vault. Every action must carry a valid agent EIP-712 signature and survive on-chain checks for registered tokens, capability status, capability expiry, intent deadline, nonce reuse, Chainlink-valued per-trade and daily limits, slippage, and vault sufficiency.

What this document does not claim

The current code does not implement a per-user token allowlist, action bitmap, rolling 24-hour window, Latch-managed signing key, or trustless GenLayer-to-Sepolia bridge. The older ARCHITECTURE.md describes several of those as planned features. This document follows the current code and deployment metadata instead.

1.1 User-visible outcome

- In copilot mode, Roque interprets and prepares the transaction, but the user's connected wallet approves and sends every on-chain action.
- In autonomous confirm mode, the user reviews the interpreted card and then asks the backend agent to act under an existing capability.
- In autonomous direct mode, a valid interpretation is passed to the agent execution path immediately, without a second UI confirmation.
- Market swaps settle against one of 45 direct constant-product pools. Limit orders escrow input in the OrderBook and settle later when the ETH/USD trigger is true.
- The activity layer reconstructs requests from Neon and settlement from Sepolia events; Sepolia remains the financial source of truth.

2. Scope, Sources, and Current Snapshot

The repository contains an early architecture plan, a newer README, current source code, deployment artifacts, and tests. Where they conflict, this analysis prioritizes executable source and the deployment JSON consumed by the running application.

Priorit y	Source	How it is used
1	contracts/src/*	Authoritative financial behavior and enforcement.
2	packages/core/src/*	Authoritative backend orchestration, signing, persistence, workers, and RPC behavior.
3	packages/genlayer/contracts/roque_interpreter.py	Authoritative natural-language interpretation and deterministic normalization.
4	packages/shared/src/deployment.json	Current addresses, token registry, feeds, and complete pool mesh.
5	Tests	Evidence for intended invariants and rejection paths.
6	README.md / ARCHITECTURE.md	Context and original intent; some deployment and feature statements are stale.

2.1 Current deployment endpoints

Component	Network	Address
DEXRouter	Sepolia	0x7965E72630cDBC3d0cc0D6Dc75497d674d49799
OrderBook	Sepolia	0x04d38a17587B4F7Ba9c477857735475aD39c61B4
AgentExecutor	Sepolia	0xffF3ACF2377C831886bF674543eb4bE38DF19a5cc
FaucetRouter	Sepolia	0x9E3d2EBb9c2dE8665f2F198b59Fe780FA2424077
Configured agent signer	Sepolia EOA	0x1254F7FBc7Cb0289c06a649A032783e7d0C68934
ETH/USD trigger feed	Sepolia Chainlink	0x694AA1769357215DE4FAC081bf1f309aDC325306
RoqueInterpreter	Genlayer Studio Network	0x0A30a949542A7EDE8a546D6Ad2C55738e8146573

Snapshot warning

The addresses above come from the files imported by the running application at commit b029735. The addresses printed in the top-level README are from an older deployment.

2.2 Repository topology

Area	Responsibility
apps/web	Next.js UI plus thin server routes over the shared core. Frontend detail is intentionally minimized here.
apps/relayer	Fastify transport and long-lived keeper/indexer loops.
packages/core	Single backend implementation shared by Next.js routes and the standalone relayer.
packages/shared	ABIs, addresses, token/pool metadata, and EIP-712 domain/type definitions.
packages/genlayer	Python intelligent contract, local runtime shim, tests, and deployment record.
contracts	Foundry project for token, AMM, router, order book, executor, faucet helper, deployment, and tests.

3. System Architecture and Trust Boundaries

Roque is best understood as five cooperating planes. Each plane owns a different kind of truth, and the design is safest when those responsibilities do not collapse into one service.

Plane	Owns	Does not own
Judgment	Semantic interpretation and optional subjective adjudication.	Funds, permissions, or execution finality.
Coordination	Request lifecycle, quoting, typed-intent construction, signatures, broadcast.	Authority to exceed a user's on-chain capability.
Authorization	Vault balances, capability, nonces, USD accounting, slippage policy.	Natural-language meaning.

Plane	Owns	Does not own
Execution	AMM reserve math, escrowed orders, trigger checks, token transfers.	Agent intent or business preference.
Governance / audit	Latch proxy policy and activity; Neon request/trade notebook.	Canonical balances or settlement state.

3.1 Data ownership

Store / system	Canonical data
Ethereum Sepolia	Token balances, pool reserves, approvals, vault balances, capabilities, nonces, daily spend, orders, fills, cancellations, and emitted settlement events.
GenLayer	Interpretation and adjudication JSON keyed by request id after validator consensus.
Neon Postgres	Natural-language request audit trail, cached interpretation, transaction hash, event-derived trade history, and indexer bookmark.
Latch	External upstream secret, scoped latch token, configured request/response policy, and allowed/denied activity trace.
Server environment	Agent signer private key, relayer gas-payer private key, database URL, GenLayer/RPC endpoints, and optional Latch token.
Browser local storage	Conversation display state and autonomous confirm/direct preference; never canonical financial state.

3.2 Network call graph

A Browser / API POST /interpret with command, mode, and optional user address.	B Core service Neon insert, Chainlink context read, GenLayer write/wait/read.	C Decision branch Copilot returns data for a user signature; autonomous constructs an agent-signed intent.	D Sepolia Executor/router/order book/pool enforce and settle.
--	---	--	---

No trustless bridge

GenLayer does not send a Sepolia transaction in this implementation. The backend waits for the GenLayer result, reads it, stores it in Neon, and later constructs a separate Sepolia transaction. The database and relayer are therefore coordination dependencies, but the AgentExecutor still bounds what their output can do.

4. Natural-Language Trade Lifecycle

This section follows a command from ingress through a structured, quoted intent. The execution branch is covered separately for copilot and autonomous mode.

4.1 Request ingress and audit record

#	Actor / boundary	Technical action	State produced
1	<code>POST /api/interpret</code>	The browser submits command, mode, and optional user address.	Untrusted HTTP body.
2	<code>handleInterpret</code>	Zod enforces a 1-500 character command, mode in copilot/autonomous, and Ethereum-address shape.	Validated input or HTTP 400.
3	<code>interpretCommand</code>	A UUID is generated and inserted into Neon with status interpreting.	Durable request id and original command.
4	<code>buildContext</code>	The backend reads ETH/USD and includes the canonical token symbol list. Price failure is tolerated.	Small hint object; never permission.

The database write occurs before the model call. This preserves the exact user command even when GenLayer fails. Neon is described in the source as the agent's notebook, not its wallet: chain state always wins for balances, orders, and

authorization.

4.2 GenLayer invocation and consensus

#	Actor / boundary	Technical action	State produced
5	<code>core/genlayer.interpret</code>	A throwaway GenLayer account submits a write to <code>RoqueInterpreter.interpret(requestId, command, contextJson)</code> .	GenLayer transaction hash.
6	Retry wrapper	Transient fetch, timeout, socket, and network errors are retried up to four attempts.	Either a submitted write or a backend failure.
7	Receipt wait	The client waits with 30 retries at four-second intervals.	Finalized/accepted GenLayer result for this client flow.
8	<code>get_interpretation</code>	A view call reads the JSON stored under the request id.	Raw JSON string parsed into Interpretation.

4.3 What the intelligent contract asks the model to produce

Field	Meaning
<code>kind</code>	swap, limit, or unknown.
<code>tokenIn / tokenOut</code>	What the user spends and receives, normalized to one of ten Roque symbols.
<code>amount</code>	Decimal string. The execution engine supports exact-input amounts only.
<code>amountToken</code>	Model annotation identifying whether the amount describes input or desired output.
<code>amountIsPercent</code>	Whether amount is a percentage of the relevant input balance.
<code>triggerPrice / triggerAbove</code>	ETH/USD threshold and direction for resting orders.
<code>confidence / reason</code>	Display metadata; not an authorization input.

4.4 Deterministic normalization inside GenLayer

1. Strip model prose or code fences and extract the first usable JSON object.
2. Require kind to be swap or limit; otherwise return a structured rejection.
3. Map loose aliases such as ETH, ether, BTC, dollars, euro, and gold to canonical r-prefixed symbols.
4. Reject unknown tokens, same-token trades, missing amounts, non-decimal amounts, and zero.
5. Require percentages to satisfy $0 < \text{percent} \leq 100$ without floating-point arithmetic.
6. Reject exact-output requests. If the user fixes the amount to receive, the interpreter asks them to restate how much input to spend.
7. Permit limit orders only when rWETH is one side, because the OrderBook watches one ETH/USD feed.
8. Require a positive trigger price for a limit order.
9. Derive buy/sell/swap from stablecoin direction instead of trusting the model's action label.
10. Return sorted JSON so validator comparison operates on normalized structure instead of prose.

Consensus mechanism

The interpreter wraps the LLM work in GenLayer `prompt_comparative`. Validators independently produce normalized intents and accept equivalence only when kind, pair, amount, percentage flag, trigger direction, and trigger price (within one percent) agree. A malformed consensus blob becomes a low-confidence rejection.

4.5 Backend validation, persistence, and quote

1. The backend parses the stored JSON. Non-JSON or absent data becomes a refusal object.
2. A rejected interpretation is written to Neon with status rejected, reason, error, and full JSON.

3. The backend independently resolves token symbols from the shared deployment registry. An unknown symbol is rejected again even if GenLayer marked it valid.
4. A valid interpretation is persisted with status ready and all structured fields.
5. For a non-percentage amount, the backend attempts a current on-chain quote through `DEXRouter.quoteSwap` and computes display notional from Chainlink.
6. Quote failure does not invalidate interpretation. The actual prepare/execute step quotes again and fails explicitly if the pool cannot serve the trade.

The quote path is deterministic: human units are converted with the input token's decimals, the router locates the pair's single pool, and `LiquidityPool.getAmountOut` applies the constant-product formula with the pool's 30 basis-point fee.

5. Copilot Execution Path

Copilot is the non-delegated path. The backend interprets and prepares, but the user wallet is the transaction sender. No capability, agent signature, `AgentExecutor` vault, or Latch relay path is required.

5.1 Copilot market swap

Exact call sequence

#	Actor / boundary	Technical action	State produced
1	Backend	<code>prepareCopilotSwap</code> re-quotes the exact input and computes <code>minAmountOut</code> from the selected slippage basis points.	Router address and raw call arguments.
2	User wallet	If allowance is below <code>amountIn</code> , approve the router. The helper requests the exact amount, not unlimited approval.	ERC-20 allowance.
3	User wallet	Call <code>DEXRouter.swapExactTokensForTokens</code> with a ten-minute deadline and the user as recipient.	Wallet-signed Sepolia transaction.
4	DEXRouter	Pull <code>tokenIn</code> from the user, approve the selected pool, and invoke <code>LiquidityPool.swap</code> .	Input reaches the AMM.
5	LiquidityPool	Calculate output, enforce <code>minAmountOut</code> , transfer output to the user, and sync reserves.	Settled balances and Swapped event.
6	Browser/backend	The browser waits for the receipt, then reports the tx hash to <code>/swap/confirm</code> .	Intent status submitted; indexer later records <code>RouterSwap</code> .

5.2 Copilot limit order

1. Percentage commands are resolved against the connected wallet balance.
2. The browser computes a trigger-aware output floor: each leg is valued in USD, the `rWETH` leg uses the trigger price, the other leg uses the current token oracle, then the selected slippage is applied.
3. The user approves the `OrderBook` for the exact input amount and calls `createOrder`.
4. `OrderBook` transfers the input into escrow, records `owner/pair/amount/minOut/trigger/expiry/status`, and emits `OrderCreated`.
5. The order expires after seven days by default. Expiry blocks fills but does not automatically return escrow; the user must cancel.

Frontend boundary only

The frontend's important architectural role is choosing which wallet signs. Copilot contract calls originate from the user; autonomous contract calls originate from the relay and carry a separate agent signature inside `calldata`.

6. Autonomous Mode in Depth

Autonomous mode changes authorization, funding, signing, gas payment, and output custody. It does not change the AMM or order-trigger contracts. The mode is built from two durable on-chain primitives: an isolated user vault and a revocable capability naming one agent signer.

6.1 Prerequisites: vault funding

1. The user selects a token and amount in token or USD terms.
2. The wallet approves AgentExecutor for the exact raw token amount if required.
3. The wallet calls AgentExecutor.deposit(token, amount).
4. AgentExecutor pulls the tokens and increments vaultBalance[user][token].
5. Only the user can call withdraw. Autonomous swaps can transform balances inside the vault, but the agent cannot deposit more from the wallet or withdraw to an arbitrary destination.

Isolation property

A capability alone is insufficient. An empty vault causes autonomous execution to revert. Conversely, vault funds without a live capability cannot be spent by the agent.

6.2 Capability grant and the first signature

The user signs an EIP-712 Grant off-chain. The shared domain is name `RoqueAgentExecutor`, version `1`, chain id `11155111`, verifying contract `0xff3ACF2377C831886bF674543eb4bE38DF19a5cc`.

Grant field	Enforced meaning
<code>user</code>	The wallet whose vault and capability are affected.
<code>agentSigner</code>	The only address whose per-action EIP-712 signatures are accepted.
<code>maxPerTradeUsd</code>	Maximum input notional for one autonomous action, 1e18 USD fixed point.
<code>maxDailyUsd</code>	Maximum input notional booked in one block.timestamp / 1 days bucket.
<code>maxSlippageBps</code>	How far minAmountOut may sit below the executor's current router quote.
<code>validUntil</code>	Unix timestamp after which all new agent actions revert.
<code>grantNonce</code>	On-chain counter preventing replay of an older signed grant.

1. The browser reads grantNonce and signs the exact typed message.
2. The signed values are sent to POST /grant.
3. The relayer pays gas and calls grantCapabilityWithSig.
4. AgentExecutor recomputes the typed-data digest using the current nonce and recovers the user address.
5. On success, grantNonce increments and the capability mapping is overwritten with the new limits and revoked=false.

Gas sponsorship does not create authority

The relayer can submit the grant transaction, but it cannot fabricate the user's EIP-712 signature. A mismatched field changes the digest and fails recovery.

6.3 Confirm versus direct autonomous execution

Mode	When POST /execute is called	Cryptographic difference
Confirm	After interpretation, when the user clicks the action on the structured intent card.	None. The click is UX confirmation, not an additional wallet signature.
Direct	Immediately after a valid interpretation if the client sees a live capability.	None. The same backend endpoint, agent key, and executor checks are used.

Critical distinction

Once the capability exists, the security boundary is the capability and vault, not the confirm-mode click. Direct mode removes the final human review but does not widen the on-chain box.

6.4 Backend autonomous execution

From ready intent to signed transaction

#	Actor / boundary	Technical action	State produced
1	POST /execute	Validate intent id, user address, and requested slippage (0-5000 bps).	Typed request.
2	Neon	Load the intent row by id and read its stored interpretation JSON.	Candidate action.
3	Shared registry	Resolve both symbols to deployed token metadata.	Addresses and decimals.
4	AgentExecutor read	Require a capability that exists, is not revoked, and names the server's agent signer.	Friendly pre-check.
5	Amount resolver	Absolute amount passes through. Percentage amount is calculated from the user's input-token vault balance.	Concrete exact input.
6	Vault read	Refuse early when balance is below amountIn.	Friendly pre-check.
7	Router quote	Quote current output and set minOut using min(requested slippage, capability max slippage).	Execution floor.
8	Nonce/deadline	Choose a millisecond timestamp nonce verified unused; set a five-minute intent deadline.	Replay and staleness controls.
9	Agent signer	Sign SwapIntent or LimitIntent using AGENT_SIGNER_PRIVATE_KEY.	Per-action EIP-712 signature.
10	Relayer wallet	Encode executeSwap or createLimitOrder and broadcast to AgentExecutor.	Sepolia tx hash returned immediately.

Broadcast versus confirmation

submitSwap, submitLimitOrder, submitGrant, and keeper fill return the transaction hash without waiting for a receipt. The UI may report success before mining. By contrast, the copilot browser helpers wait for receipts. The indexer is the later evidence that an autonomous action actually emitted settlement events.

6.5 The second signature: per-action agent intent

Intent	Signed fields
SwapIntent	user, tokenIn, tokenOut, amountIn, minAmountOut, nonce, deadline
LimitIntent	user, tokenIn, tokenOut, amountIn, minAmountOut, triggerPrice, triggerAbove, expiry, nonce, deadline

The relayer transaction sender and the agent intent signer are separate roles. The relayer pays gas and delivers calldata. AgentExecutor ignores the transaction sender for capability authorization and instead recovers the EIP-712 signer from the inner signature.

6.6 On-chain authorization order

1. Reject zero input.
2. Reject an expired per-action deadline.

3. Require both tokenIn and tokenOut to be globally registered.
4. Load the user's capability and require exists, not revoked, and not expired.
5. Recover the EIP-712 signer and require equality with capability.agentSigner.
6. Reject a used nonce, then mark the nonce used. Any later revert rolls this state change back atomically.
7. Value tokenIn amount in 1e18 USD. Stables are fixed at one dollar; other assets use the registered Chainlink feed and token/feed decimals.
8. Reject value above maxPerTradeUsd.
9. Add value to dailyUsdSpent[user][block.timestamp / 1 days] and reject totals above maxDailyUsd.
10. For swaps, read a fresh router quote and reject minAmountOut below the capability's slippage floor.
11. Debit the user's vault and proceed to the typed execution path.

Daily means epoch day, not rolling day

The accounting key is integer division of block.timestamp by one day. Spend resets at the UTC day boundary. It is not a trailing 24-hour window.

6.7 Autonomous market-swap settlement

1. AgentExecutor debits vaultBalance[user][tokenIn].
2. It grants the router an exact allowance with forceApprove.
3. The router pulls tokenIn from AgentExecutor and forwards it into the selected pool.
4. LiquidityPool computes amountOut with $x*y=k$ and a 30 bps input fee, then checks minAmountOut.
5. The pool transfers tokenOut back to AgentExecutor and syncs reserves to real balances.
6. AgentExecutor credits vaultBalance[user][tokenOut] and emits AgentSwap with the Chainlink USD value.

Output custody

An autonomous market swap keeps the output inside the user's executor vault. The user must withdraw it with their own wallet to return it to the main wallet.

6.8 Autonomous limit-order creation and later fill

1. The same authorization path books the action against per-trade and daily USD caps at order creation time.
2. AgentExecutor debits the vault and approves OrderBook.
3. OrderBook.createOrderFor pulls the input, records the user as owner, and escrows the funds.
4. The current backend computes minAmountOut from the pool quote at creation time, unlike the copilot builder's trigger-aware oracle calculation.
5. The order receives a seven-day expiry and an ETH/USD trigger scaled to eight decimals.
6. A later keeper or any third party calls executeOrder only when it believes the trigger is met.
7. OrderBook independently rechecks open status, expiry, Chainlink positivity, maximum staleness, and trigger direction.
8. The order is marked Filled, the router swap executes, and output is sent directly to the order owner. If the router reverts, the entire fill including the status change reverts.

Important custody and revocation behavior

Autonomous limit-order output goes to the user's wallet, not back to the executor vault. Revoking the capability stops new agent actions but does not cancel an already escrowed order. That order can still fill until expiry or user cancellation.

7. Solidity Contract Architecture

7.1 TestToken and FaucetRouter

Each of the ten tradable assets is an OpenZeppelin ERC-20 with configured decimals and a metered faucet. A faucet pull is sized near USD 1,000 using deployment-time prices and is capped at five claims per address. FaucetRouter batches all token faucets in one transaction and catches individual failures so exhausted assets do not revert the entire batch.

7.2 Complete pool mesh

The deploy script creates one pool for every unordered pair of ten assets: $10 * 9 / 2 = 45$ pools. Every pair therefore routes in one hop. Each side is seeded with approximately USD 1,000,000 at the deployment-time Chainlink price, and every pool charges 30 basis points.

Property	Implementation
Curve	$x * y = k$ constant-product AMM.
Quote	<code>amountInWithFee * reserveOut / (reserveIn * BPS + amountInWithFee)</code>
Liquidity shares	Internal share accounting; first deposit locks MINIMUM_LIQUIDITY.
Token order	Canonical address sorting into token0/token1.
Reserve updates	Explicit accounting for liquidity; post-swap sync from actual ERC-20 balances.
Reentrancy	<code>addLiquidity</code> , <code>removeLiquidity</code> , and <code>swap</code> are nonReentrant.

7.3 DEXRouter

- Ownable registry maps an order-independent pair key to one pool.
- `quoteSwap` delegates directly to the pool's `getAmountOut`.
- `swapExactTokensForTokens` enforces a deadline and nonzero recipient.
- The router is intentionally thin and does not maintain balances between completed calls.
- All user, executor, and order-book swaps converge on this same code path and emit `RouterSwap`.

7.4 OrderBook state machine

State	Entry	Exit
None	Unallocated id.	<code>createOrder</code> or <code>createOrderFor</code> -> Open.
Open	Input escrowed; owner and trigger recorded.	<code>executeOrder</code> -> Filled, <code>cancelOrder</code> -> Cancelled.
Filled	Trigger and router swap succeeded atomically.	Terminal.
Cancelled	Escrow returned to owner.	Terminal.

- Only the configured `AgentExecutor` can call `createOrderFor`.
- Anyone can call `executeOrder`; the caller has no pricing authority.
- The single configured price feed is ETH/USD. Interpreter and browser code restrict limit intents to pairs containing `rWETH`.
- A feed answer older than `maxPriceStaleness` (24 hours by default) blocks execution.
- Only the owner or `AgentExecutor` address may cancel, but the shipped `AgentExecutor` exposes no cancel wrapper, so normal autonomous cancellation is user-signed.

7.5 AgentExecutor as the authorization choke point

`AgentExecutor` combines token valuation, isolated custody, delegated authority, replay protection, spend accounting, and typed dispatch. Its security property comes from having only two agent action entry points: `executeSwap` and `createLimitOrder`. There is no generic `execute(target, calldata)` function.

State mapping	Purpose
<code>tokenInfo[token]</code>	Global registration, decimals, stable flag, and Chainlink feed.
<code>vaultBalance[user][token]</code>	Only token inventory available to autonomous actions.
<code>capabilities[user]</code>	Current signer and policy bounds.
<code>usedNonce[user][nonce]</code>	One-time per-action replay protection.
<code>grantNonce[user]</code>	One-time capability-signature replay protection.
<code>dailyUsdSpent[user][day]</code>	Input notional already booked in the current epoch day.

7.6 Administrative trust

The contracts are not fully governance-minimized. DEXRouter owner can register pair mappings; OrderBook owner can replace the price feed, staleness threshold, and agent executor; AgentExecutor owner can register token valuation metadata; TestToken owner can mint. Those powers are appropriate for a testnet demo but must be included in a production threat model.

8. Keeper, Indexer, and Database

8.1 Keeper loop

1. Read nextOrderId.
2. Iterate every historical id from 1 to nextOrderId - 1.
3. Read each order and skip non-open states.
4. Call isTriggered for every open order.
5. For triggered orders, encode executeOrder(id) and broadcast from the relayer wallet.
6. Record hash or error in the tick result; retry naturally on the next 15-second loop.

Keeper safety and liveness

The keeper cannot force an untriggered fill because OrderBook repeats the oracle check. If the keeper is offline or Latch denies its RPC call, funds remain escrowed and the order waits. The present scan is $O(\text{total orders})$, including terminal orders, and will need event- or index-based optimization at larger scale.

8.2 Indexer loop

- Starts at the configured AgentExecutor deployment block and scans at most 800 blocks per window.
- Reads AgentSwap, OrderCreated, OrderFilled, OrderCancelled, and RouterSwap in parallel.
- Skips RouterSwap events whose sender is AgentExecutor or OrderBook to avoid double-counting.
- Fetches each event block timestamp once and inserts normalized trade rows.
- Uses `UNIQUE(tx_hash, log_index)` plus `ON CONFLICT DO NOTHING` for idempotence.
- Persists last_block in indexer_state and catches up to head every 20 seconds.

Finality caveat

The indexer scans to the current head with no confirmation depth and does not implement reorg rollback. This is acceptable for a Sepolia demo but not a complete production indexer.

8.3 Intent and trade records

Table	Role	Current lifecycle
intents	Original command, parsed fields, status, errors, tx hash.	interpreting -> rejected/ready -> submitted or failed. signed and confirmed are allowed by schema but not currently set.
trades	Event-derived flat history for activity display.	Inserted only from Sepolia logs; never authoritative over chain state.
indexer_state	Restart-safe block bookmark.	Advanced after each successfully scanned window.

8.4 Two backend hosting shapes

Shape	Behavior
Next.js server routes	Thin route files call the same core handlers. Keeper/indexer run through secret-guarded cron endpoints.
Fastify relayer	Thin HTTP shell over core handlers. Starts in-process keeper and indexer loops after listen.

9. Latch's Role in Autonomous Mode

In the current repository, Latch is an optional outbound RPC boundary. It is activated only when both `LATCH_RPC_URL` and `LATCH_TOKEN` are set. The viem wallet still signs locally with the relayer private key; its transport then sends the JSON-RPC request to Latch with Authorization: Bearer lat_... instead of sending directly to the configured Sepolia RPC.

9.1 Exact placement

1 Local construction Core encodes AgentExecutor or OrderBook calldata.	2 Local signing Viem signs with the relayer gas-payer private key.	3 Latch proxy Bearer token identifies upstream and policy; request may be allowed or denied.	4 Sepolia RPC Allowed raw transaction is broadcast; the chain may mine or revert it.
--	--	--	--

Traffic	Uses Latch when configured?
Autonomous market-swap broadcasts	Yes
Autonomous limit-order creation broadcasts	Yes
Gas-sponsored capability grant broadcasts	Yes
Keeper executeOrder broadcasts	Yes
Sepolia reads and quotes	No; direct publicClient transport
Copilot user-wallet transactions	No; sent through the connected wallet provider
GenLayer writes/reads	No
Agent EIP-712 signature creation	No; local AGENT_SIGNER_PRIVATE_KEY

9.2 What Latch contributes

- Credential separation: the relayer can hold a scoped latch token while the real upstream credential remains stored and injected by Latch.
- Independent revocation: deleting or disabling a latch can stop proxy access without rotating the upstream provider key.
- Ordered request policy: endpoint, HTTP method, payload, body size, time window, IP, identity, rate, custom code, and related filters can deny before the upstream is called.
- Activity trace: allowed and denied calls are recorded with the filter decision path; request/response body logging is optional.
- Operational containment: a compromised process that has only the latch token is limited by the external policy rather than inheriting the full upstream credential.

- Incremental integration: Roque changes only the viem transport URL and Authorization header; contract logic remains independent.

9.3 What Latch does not do in this code

- It does not hold or use AGENT_SIGNER_PRIVATE_KEY.
- It does not hold or use the relayer private key; viem signs locally before proxying.
- It does not validate the user's on-chain capability. AgentExecutor does.
- It does not make a transaction final or successful. It only controls whether a request reaches the upstream RPC.
- The repository contains no checked-in Latch pipeline definition, so active filters, rate limits, identities, and spend policy cannot be verified from source.
- A stolen local private key can broadcast through another RPC unless infrastructure prevents egress. Latch is not key custody in the present integration.

Fine-grained EVM policy needs more work

The proxied write is normally `eth_sendRawTransaction` with opaque signed bytes in `params[0]`. A payload filter can allow only the JSON-RPC method, but contract/function/value allowlisting requires transaction decoding in custom policy or a redesigned Latch/TEE signing path. A model-based `spend_limit` is not automatically an Ethereum gas or token-spend limit.

9.4 Recommended Latch policy for Roque

1. Bind the production RPC credential as a write-only Secret and issue a dedicated latch for Roque's relayer.
2. Allow only HTTP POST and the exact JSON-RPC proxy path used by the provider.
3. Require JSON-RPC 2.0 and allow `$.method` only as `eth_sendRawTransaction` for the write latch.
4. Set a conservative request rate compatible with autonomous execution and keeper cadence.
5. Apply body-size limits and IP allowlisting or hardware-bound identity for the relayer host.
6. Disable request-body logging if raw signed transactions are considered sensitive operational data; keep decision metadata.
7. Use a separate read latch only if protected provider reads are needed. The current application reads directly.
8. For real transaction-content controls, add audited transaction decoding/custom code or move signing into an enclave-backed EVM signer design.

10. Why Rialo/Latch Helps Roque Users

Roque's concrete Rialo-facing integration today is Latch. Its value is not another copy of the on-chain capability; it is a separate control plane around the off-chain machinery that uses credentials and broadcasts transactions.

User benefit	Technical reason
Smaller off-chain blast radius	A scoped, revocable proxy token can replace direct possession of the upstream credential.
Defense in depth	Latch can reject the relayer request before RPC while AgentExecutor independently rejects invalid financial actions on-chain.
Auditable automation	Latch records attempted access decisions; Sepolia records accepted transactions and settlement events.
Faster operational shutdown	Operators can revoke proxy access immediately while users retain the separate on-chain revoke path.
Safer always-on behavior	Rate, time, identity, endpoint, and payload policy can contain runaway relayer/keeper traffic.
Better UX without surrendering custody	Gas-sponsored grants and autonomous broadcasts avoid repeated wallet popups while the vault and contract caps retain hard limits.
Graceful degradation	Copilot user-signed trading does not depend on Latch or the relayer wallet path.

10.1 Architectural fit with broader Rialo capabilities

The current settlement chain is Sepolia and the current timer is an off-chain keeper. Rialo's published direction around infrastructure for intelligent systems, configurable privacy, and reactive transactions is relevant as an evolution path, not as a description of deployed Roque behavior.

- Native reactive automation could replace or reduce the centralized 15-second keeper loop for time/event-triggered orders.
- Configurable privacy could protect sensitive mandates, policy parameters, or external evidence while preserving verifiable execution.
- A native confidential execution/signing path could remove raw agent and relayer keys from the application environment.
- Moving policy and automation closer to the execution environment could reduce the number of independently operated services required for an autonomous order.

Implemented versus potential

None of the broader Rialo-native items above are present in this repository snapshot. They are concrete architecture opportunities inferred from Roque's current keeper, credential, privacy, and cross-system coordination needs.

11. Security Model and Failure Analysis

11.1 Enforced invariants

Invariant	Enforcer
Only registered tokens can enter an autonomous intent.	AgentExecutor tokenInfo checks; shared registry and interpreter also validate earlier.
Only the configured agent signer authorizes an action.	EIP-712 recovery against capability.agentSigner.
One signed intent cannot be replayed.	usedNonce per user.
Stale action signatures expire.	Intent deadline.
Delegation expires or can be revoked.	Capability validUntil and revoked.
Input notional is bounded per action and day.	Chainlink usdValue and dailyUsdSpent.
The agent cannot accept arbitrary slippage.	Fresh router quote and capability maxSlippageBps.
The agent cannot spend the main wallet.	Vault-only debit and user-only deposit/withdraw.
Keeper cannot force an untriggered order.	OrderBook oracle, expiry, and status checks.
AMM output cannot fall below signed/constructed floor.	LiquidityPool minAmountOut check.

11.2 Threat and failure matrix

Failure / compromise	Effect	Residual control
LLM misreads command	May produce a wrong but normalized intent.	User review in copilot/confirm; vault and capability caps in autonomous.
GenLayer unavailable	Interpretation fails after retries.	Direct contract functionality and existing order settlement remain; no new chat intent.
Neon unavailable	Intent creation/execution history fails.	Sepolia funds remain unaffected; autonomous endpoint depends on stored intent.
Database modified	Stored interpretation can be changed before execution.	On-chain caps still bound value, but GenLayer provenance is not cryptographically bound to calldata.
Agent signer stolen	Attacker can produce valid per-action signatures.	Capability, vault, limits, deadlines, and nonces remain.
Relayer key stolen	Attacker can spend relayer gas and broadcast arbitrary public calls.	Cannot debit vault without a valid agent signature; Latch is bypassable through another RPC if the key is exfiltrated.
Latch token stolen	Attacker can use the configured proxy within its policy.	Real upstream secret is not disclosed; policy, identity, rate, and revocation can constrain access.
Latch unavailable/denies	Autonomous broadcasts, gas-sponsored grants, and keeper fills fail.	Copilot user-wallet writes and direct reads bypass Latch; escrow remains safe.
Keeper offline	Triggered orders wait.	Anyone may execute; contract checks still protect.
Chainlink stale/bad	Executor valuation or order fill reverts.	Fail-closed behavior; no fallback oracle in contracts.
Pool moves after quote	Swap may receive less or revert.	minAmountOut and deadline enforce the user's/agent's floor.

11.3 Current implementation risks and production work

- POST /execute is unauthenticated, loads an intent only by id, and does not verify that the intent belongs to the supplied user or was created in autonomous mode.
- The same ready intent can be executed repeatedly at the application layer because every call receives a fresh on-chain nonce. The nonce blocks replay of one signed payload, not repeated re-signing of one database intent.
- Activity endpoints expose intent ids by user address. Combined with the previous point, ownership checks and transactional status locking are production requirements.
- Autonomous submission does not wait for a receipt, and intent status is not advanced to confirmed by the indexer.
- The copilot confirm endpoint accepts a transaction hash without verifying sender, calldata, receipt status, or event correspondence.
- Raw agent and relayer private keys remain environment secrets. Latch currently protects RPC egress, not signing custody.
- Capability has no per-user token allowlist or action bitmap. Any globally registered pair and either implemented action are available within numeric bounds.
- Daily spend is a UTC epoch-day bucket. Capability replacement does not clear already booked spend for that day.
- Creating a limit order consumes daily capacity immediately even if it never fills.
- Revoking a capability does not cancel open orders already in OrderBook escrow.
- The autonomous limit minOut is based on the current pool quote, while the copilot limit builder derives a trigger-aware oracle floor. The two modes can behave differently at fill time.
- OrderBook supports only one ETH/USD trigger feed and allows a 24-hour price age by default.
- Keeper complexity grows with every historical order id, and the indexer lacks reorg reconciliation.
- The GenLayer adjudicate function exists but is not wired into the live trade lifecycle; fuzzy news/event mandates are therefore not currently autonomous features.
- buildContext passes token symbols and ETH/USD only, not user balances, positions, or block-pinned Sepolia state described in older plans.
- Contract-owner powers and test-token minting are central administrative trust assumptions.

Highest priority hardening

Authenticate execution requests, bind intent ownership and mode, atomically transition ready to executing, reject already-submitted intents, wait for receipts, bind the executed payload to the finalized GenLayer result, and move signing into a non-exportable policy-controlled key service.

12. Operational Behavior and Configuration

12.1 Required server configuration

Variable	Purpose
SEPOLIA_RPC_URL	Direct read RPC and write fallback when Latch is disabled.
AGENT_SIGNER_PRIVATE_KEY	Signs SwapIntent and LimitIntent typed data.
RELAYER_PRIVATE_KEY or DEPLOYER_PRIVATE_KEY	Signs and pays gas for backend Sepolia transactions.
DATABASE_URL	Neon intent notebook, trade cache, and indexer state.
GENLAYER_RPC_URL / GENLAYER_CONTRACT_ADDRESS	Intelligent-contract write/wait/read path.
LATCH_RPC_URL / LATCH_TOKEN	Optional replacement transport for relayer wallet broadcasts.
CRON_SECRET	Guards Next.js keeper and indexer routes.

12.2 Retry and polling behavior

Subsystem	Behavior
GenLayer network call	Up to four attempts for transient errors; receipt polling 30 times every four seconds.
Neon query	Up to four attempts with increasing 250 ms delay for transient network errors.
Market/capability UI reads	Polling intervals range from 12 to 20 seconds.
Standalone keeper	Every 15 seconds, with errors isolated to the tick.
Standalone indexer	Every 20 seconds, catches up in 800-block windows.
Next.js cron	Secret-guarded routes with maxDuration 60 seconds; external schedule configured by deployment.

12.3 Completion semantics by action

Action	When caller receives success
Copilot swap / limit / vault / revoke	After publicClient.waitForTransactionReceipt returns.
Autonomous swap / limit	After sendTransaction returns a hash; not necessarily mined.
Gas-sponsored grant	After sendTransaction returns a hash; capability may not be readable yet.
Keeper fill	After sendTransaction returns a hash; later tick/indexer reflects result.
Trade history	After indexer observes the relevant event and inserts it into Neon.

13. Testing and Verification Evidence

Verification was run against repository snapshot [b029735](#) on August 14, 2026. The documentation generator does not modify application or contract code.

Suite	Result	Coverage signal
Workspace TypeScript	All four workspace projects passed.	Shared types, backend, relayer, and web compile together.
Core Vitest	9 passed.	Slippage math, trigger scaling, and EIP-712 recovery/signature sensitivity.

Suite	Result	Coverage signal
GenLayer pytest	28 passed.	Normalization happy paths, aliases, exact-output rejection, malformed model output, limits, and adjudication fallback.
Foundry	60 passed across 6 suites.	AMM, router, order book, executor caps/replay/revoke/expiry/slippage/vault, faucet, and fuzz properties.

13.1 Particularly important tested rejection paths

- Wrong agent signer.
- Per-trade and daily-cap excess.
- Reused nonce.
- Revoked or expired capability.
- Expired action intent.
- Excessive slippage.
- Missing capability or insufficient vault.
- Untriggered, expired, stale-price, double-filled, and unauthorized-cancel limit orders.
- Unknown token, self trade, zero/non-numeric amount, out-of-range percentage, unsupported limit pair, and exact-output language.

14. Token and Oracle Registry

Symbol	Decimals	Stable	Token address	Valuation source
rUSDC	6	Yes	0x87d4794c85A739736d08F0D93e47Fb7714DaeD30	Fixed USD 1
rUSDT	6	Yes	0xbF7ee0178Ce97CF89D0af978D43081CD6d16Cb2	Fixed USD 1
rDAI	18	Yes	0x58874c7F6157E3B5AA877Aa6658bf37AAA97D674	Fixed USD 1
rWETH	18	No	0xb7F9b5Fd07e206D4139851aAa3be98348CdAa571	0x694AA1769357215DE4FAC081bf1f309aDC325306
rWBTC	8	No	0x5DB1dcc89f65D831E15e05678F2eF3bFFd3Cf344	0x1b44F3514812d835EB18DB0acB33d3fA3351Ee43
rLINK	18	No	0x22cA1B6B3dFA09B65b44700cfeE60923bcDd7D85	0xc59E3633BAAC79493d908e63626716e204A45EdF
rSNX	18	No	0x1AdD4cf8A855CA632Ed16E6c62B109EF27CF376F	0xc0F82A46033b8BdBA4Bb0B0e28Bc2006F64355bC
rFORTH	18	No	0x2ca8B7BC0F9020bC35d40be6530eAac04924B619	0x070bf128E88A4520b3EfA65AB1e4Eb6F0F9E6632
rEURC	6	No	0xd8f58240877aA3F5aD308F87CabCE7c88E656278	0x1a81afB8146aeFFcFc5E50e8479e826E7D55b910
rPAXG	18	No	0x0Da356d851ad9122D9ec4171d5983Ec3BB1B3FCf	0xC5981F461d74c46eB4b0CF3f4Ec79f025573B0Ea

Every unordered pair has one deployed pool. The complete symbol/address matrix is stored in [packages/shared/src/deployment.json](#) and is zipped into typed metadata by [packages/shared/src/index.ts](#).

15. Code Map for Maintainers

Lifecycle concern	Primary files / functions
HTTP validation and handlers	packages/core/src/api.ts
Request lifecycle and autonomous orchestration	packages/core/src/services.ts
GenLayer client and retries	packages/core/src/genlayer.ts
Agent signatures and relayer submissions	packages/core/src/intents.ts
Sepolia and Latch transports	packages/core/src/chain.ts , env.ts
Quotes and slippage	packages/core/src/quote.ts
Chainlink valuation	packages/core/src/prices.ts
Keeper	packages/core/src/keeper.ts
Indexer	packages/core/src/indexer.ts
Neon schema and query retries	packages/core/src/db/index.ts
Natural-language contract	packages/genlayer/contracts/roque_interpreter.py
Capability and vault enforcement	contracts/src/AgentExecutor.sol
Resting orders and oracle trigger	contracts/src/OrderBook.sol
Router and AMM	contracts/src/DEXRouter.sol , LiquidityPool.sol
Current shared addresses/types	packages/shared/src/index.ts , deployment.json

16. External Technical References

These references were used only to validate platform semantics around GenLayer consensus, Latch proxy policy, and Rialo's published direction. Repository code remains the source of truth for what Roque currently implements.

Topic	Official source
GenLayer intelligent contracts	https://docs.genlayer.com/developers/intelligent-contracts/introduction
GenLayer equivalence principle	https://docs.genlayer.com/developers/intelligent-contracts/equivalence-principle
Latch quickstart	https://onlatch.com/docs/get-started/quickstart
Latch secrets and scoped tokens	https://onlatch.com/docs/get-started/secrets https://onlatch.com/docs/get-started/latches
Latch policy pipeline and filters	https://onlatch.com/docs/filters/pipeline https://onlatch.com/docs/filters/reference
Latch activity and proxy API	https://onlatch.com/docs/guides/activity https://onlatch.com/docs/reference/proxy-api
Rialo overview	https://www.rialo.io
Rialo reactive transactions	https://www.rialo.io/posts/reactive-transactions-a-model-for-native-automation-on-rialo

Bottom line

Roque's core architectural achievement is not that an AI can trade. It is that language understanding, delegated permission, off-chain operations, and token settlement are split across independent boundaries. The user experience becomes simpler while the financial authority remains explicit, revocable, measurable, and enforced by contracts.